KafkaTutorial Required

9. Add Postgres and Debezium (Kafka Connect) to Docker Compose

We'll add two services:

- **postgres** – a plain Postgres with logical decoding enabled
- **connect** – Kafka Connect with Debezium Postgres connector pre-installed

Note: For Debezium/Connect to create internal/topics automatically, we'll flip the broker's `auto.create.topics.enable` to **true** (OK for labs).

Update your existing **`docker-compose.yml`** to the following (or merge these services into it):

```
version: "3.8"

services:
  kafka:
    image: apache/kafka:latest
    container_name: kafka
    restart: unless-stopped
    ports:
      - "9092:9092"     # client access
      - "9093:9093"     # controller listener
    environment:
      # --- KRaft essentials ---
      KAFKA_PROCESS_ROLES: "broker,controller"
      KAFKA_NODE_ID: "1"
      KAFKA_CONTROLLER_QUORUM_VOTERS: "1@kafka:9093"

      # --- Listeners ---
      KAFKA_LISTENERS: "PLAINTEXT://:9092,CONTROLLER://:9093"
      KAFKA_ADVERTISED_LISTENERS: "PLAINTEXT://localhost:9092"
      KAFKA_CONTROLLER_LISTENER_NAMES: "CONTROLLER"
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
"PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT"
      KAFKA_INTER_BROKER_LISTENER_NAME: "PLAINTEXT"

      # --- Lab-friendly settings ---
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"   # ✅ let Debezium & Connect
auto-create topics
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: "1"
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: "1"
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: "1"
    volumes:
      - kafka_data:/var/lib/kafka/data
    healthcheck:
      test: ["CMD-SHELL", "/opt/kafka/bin/kafka-topics.sh --bootstrap-
server localhost:9092 --list >/dev/null 2>&1"]
      interval: 10s
      timeout: 5s
      retries: 8

  postgres:
    image: postgres:15-alpine
```

```
    container_name: postgres
    restart: unless-stopped
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: appdb
    command: >
      postgres
      -c wal_level=logical
      -c max_wal_senders=10
      -c max_replication_slots=10
    ports:
      - "5432:5432"
    volumes:
      - pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d appdb"]
      interval: 10s
      timeout: 5s
      retries: 10

  connect:
    image: quay.io/debezium/connect:2.7
    container_name: connect
    restart: unless-stopped
    depends_on:
      - kafka
    ports:
      - "8083:8083"  # Kafka Connect REST API
    environment:
      BOOTSTRAP_SERVERS: "kafka:9092"
      GROUP_ID: "connect-cluster"
      CONFIG_STORAGE_TOPIC: "connect-configs"
      OFFSET_STORAGE_TOPIC: "connect-offsets"
      STATUS_STORAGE_TOPIC: "connect-status"
      KEY_CONVERTER: "org.apache.kafka.connect.json.JsonConverter"
      VALUE_CONVERTER: "org.apache.kafka.connect.json.JsonConverter"
      KEY_CONVERTER_SCHEMAS_ENABLE: "false"
      VALUE_CONVERTER_SCHEMAS_ENABLE: "false"
      # Optional: reduce noisy logs
      # LOG_LEVEL: "INFO"
    healthcheck:
      test: ["CMD-SHELL", "curl -sf http://localhost:8083/connectors
>/dev/null || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 10

volumes:
  kafka_data:
  pg_data:
```

Bring everything up (clean):

```
docker compose down -v
docker compose pull
docker compose up -d
docker compose ps
```

When all are **Up** (and healthy), proceed.

10. Create a Demo Table in Postgres

We'll create a simple table that Debezium will watch.

```
# Enter psql
docker exec -it postgres psql -U app -d appdb

-- Inside psql, run:
CREATE SCHEMA IF NOT EXISTS demo;

CREATE TABLE demo.customers (
  id            SERIAL PRIMARY KEY,
  email         TEXT UNIQUE NOT NULL,
  full_name     TEXT NOT NULL,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- A simple update trigger to touch updated_at (nice for CDC demos)
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_set_updated_at
BEFORE UPDATE ON demo.customers
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- Insert a few starter rows
INSERT INTO demo.customers (email, full_name) VALUES
('ada@example.com',  'Ada Lovelace'),
('alan@example.com', 'Alan Turing');

\q
```

11. Register the Debezium Postgres Connector

We create a connector via **Kafka Connect REST** (port `8083`).

Create a file **`postgres-connector.json`** (anywhere—e.g., in your project root):

```
{
  "name": "pg-customers-connector",
  "config": {
    "connector.class":
"io.debezium.connector.postgresql.PostgresConnector",
    "tasks.max": "1",

    "database.hostname": "postgres",
    "database.port": "5432",
    "database.user": "app",
```

```
    "database.password": "app",
    "database.dbname": "appdb",

    "topic.prefix": "pg",                      // topics will look like:
pg.demo.customers
    "publication.autocreate.mode": "filtered",
    "slot.name": "pg_slot_customers",

    "schema.include.list": "demo",           // limit CDC to our schema
    "table.include.list": "demo.customers",  // and table
    "tombstones.on.delete": "false",

    // Converters (we set Json w/o schemas in the Connect service)
    "decimal.handling.mode": "string",
    "time.precision.mode": "adaptive_time_microseconds",

    // Heartbeats & polling
    "heartbeat.interval.ms": "3000",
    "poll.interval.ms": "1000"
  }
}
```

Register the connector:

```
curl -i -X POST http://localhost:8083/connectors \
  -H "Content-Type: application/json" \
  --data @postgres-connector.json
```

Verify it's running:

```
curl -s http://localhost:8083/connectors | jq
curl -s http://localhost:8083/connectors/pg-customers-connector/status | jq
```

Expected: `RUNNING`.

What topics get created?

- `pg.demo.customers` – change events (insert/update/delete) for your table
- `connect-*` – Kafka Connect internal topics
  (Auto-created because we enabled `KAFKA_AUTO_CREATE_TOPICS_ENABLE=true`.)

---

12. Produce a Change and Watch Kafka

Make a new change in Postgres and read it in your existing **Python consumer** (from point 7). Either:

- Start your **consumer** and set the topic to `pg.demo.customers`, or
- Use a one-off console consumer.

A) Use your Python consumer

In **`consumer.py`**, temporarily set:

```
TOPIC = "pg.demo.customers"
```

Run it:

```
cd kafka-lab/app
source venv/bin/activate
python consumer.py
```

B) Or use Kafka's console consumer (quick peek)

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic pg.demo.customers \
  --from-beginning
```

Now perform a change in Postgres:

```
docker exec -it postgres psql -U app -d appdb

-- INSERT
INSERT INTO demo.customers (email, full_name)
VALUES ('grace@example.com', 'Grace Hopper');

-- UPDATE
UPDATE demo.customers
   SET full_name = 'Grace B. Hopper'
 WHERE email = 'grace@example.com';

-- DELETE (optional to observe deletes)
DELETE FROM demo.customers WHERE email = 'alan@example.com';
\q
```

You should see Debezium emit **change events** in JSON (with `op` = `c` create, `u` update, `d` delete). Your consumer will print them.

---

13. Understanding the Debezium Event

A typical **INSERT** event (simplified) looks like:

```
{
  "before": null,
  "after": {
    "id": 3,
    "email": "grace@example.com",
    "full_name": "Grace Hopper",
    "created_at": "2025-11-09T16:40:20.123456Z",
    "updated_at": "2025-11-09T16:40:20.123456Z"
  },
  "source": {
    "db": "appdb",
    "schema": "demo",
    "table": "customers",
    "lsn": 123456789
  },
```

```
  "op": "c",
  "ts_ms": 1731160820123
}
```

- `before` – row state **before** the change (null for inserts)
- `after` – row state **after** the change
- `op` – operation type: `c` (create), `u` (update), `d` (delete), `r` (snapshot)
- `source` – metadata about where the change came from

---

14. Common Pitfalls & Fixes

- **Connector won't start / errors about slots or publications**
  - Ensure Postgres is running with `wal_level=logical` (we set it in `command:`).
  - Make sure credentials/host/port/db match the container names and envs.
- **No events appear**
  - Confirm the connector is **RUNNING** (`/status` endpoint).
  - Ensure you changed data **after** the connector started (unless you configure snapshots).
  - Check that you're consuming the correct topic: `pg.demo.customers`.
- **Topic not found**
  - We enabled `KAFKA_AUTO_CREATE_TOPICS_ENABLE=true`. If you keep it **false**, you must pre-create:
    - `connect-configs`, `connect-offsets`, `connect-status`
    - `pg.demo.customers`
    - (and possibly Debezium's schema changes topic if configured)
- **Windows/WSL networking quirks**
  - Keep Docker and your Python app on the same side (both Windows host or both WSL). Using `localhost:9092` from the host is typically fine.

---

15. Clean Up

```
# Stop everything and remove volumes (data!)
docker compose down -v
```

If you only want to stop services but **keep** data:

```
docker compose down
```